

EXPERIMENT 3-DEBUGGING C PROGRAMS

NAME	PRABHU DAYAL NAGPAL
BATCH	16
SAP ID	590033396

1. Fundamentals of Errors and Debugging

Key Definitions

- Error: A mistake or blunder made by the programmer in the source code.

Example: Writing $a + b$ when the intention was to multiply $a * b$.

- Bug: A flaw or fault in a computer program that causes it to produce an incorrect or unexpected result, or to

behave in unintended ways.

Example: The program outputs the sum of two numbers instead of their product.

- Debugging: The systematic process of finding, analyzing, and removing bugs or defects in a computer

program.

2. Classifications of Errors in C

Error Type

Description

Primary Cause

Example

Syntax Errors

Code violates the grammatical rules of the C language.

Missing semicolons, unmatched braces, typos in keywords.

```
int a = 10 (missing  
;)
```

Compilation /

Type Errors

Operations performed on incompatible types or missing declarations.

Assigning float string to int pointer, using undeclared variable.

```
int *p = "hello";
```

Linker Errors

The compiler generates object files, but the linker cannot combine them.

Function declared but not defined, missing libraries.

undefined reference

to 'main'

Runtime Errors The program crashes or terminates abnormally during execution.

Division by zero, accessing invalid memory addresses.

```
int x = 10 / 0;
```

Logical Errors

Code runs to completion without crashing, but output is incorrect.

Wrong mathematical formula, incorrect loop condition.

```
for(i=0; i<5; i++)
```

instead of `<=5`

Why Does a Program Compile Successfully but Produce Incorrect Output?

A compiler's job is to verify that your code obeys the syntax and type rules of the C language, and to convert

that code into machine instructions. The compiler cannot understand human intent.

- If you write `sum = a - b;` instead of `sum = a + b;` the statement is completely valid C syntax. Therefore,

compilation succeeds, but the resulting logic produces an erroneous output at execution time.

Case 1: Syntax & Compilation Errors

When the compiler encounters invalid code, it prints diagnostics specifying the file name, line number, column,

and error description.

Example Code (sample1.c)

```
#include <stdio.h>

int a = 10

printf("%d\n", a);

return 0;

}
```

Compilation Command & Output

```
$ cc sample1.c
```

```
sample1.c:5:5: error: expected ',' or ';' before 'printf'
```

Breakdown of Error Message

- Filename: sample1.c
- Line & Column: Line 5, Column 5
- Type of Message: error
- Description: Expected , or ; before printf statement.

Case 2: Compiler Warnings (-Wall, -Wextra)

A warning indicates code that is syntactically legal, but potentially dangerous or suspicious. Warnings do not

stop compilation, but ignoring them frequently leads to bugs at runtime.

Example Code (sample2.c)

```
#include <stdio.h>

int main()
```

```
{  
  
int a;  
  
printf("Hello\n");  
  
return 0;  
  
}
```

Standard Compilation vs. Warning Flags

```
# Standard compilation (may suppress minor warnings)  
  
$ cc sample2.c  
  
$ ./a.out
```

Hello

```
# Enabling comprehensive warning flags  
  
$ cc -Wall -Wextra sample2.c
```

sample2.c:5:9: warning: unused variable 'a' [-Wunused-variable]

- -Wall: Enables a broad set of commonly useful compiler warnings.
- -Wextra: Enables additional warnings not covered by -Wall.

Case-3: printf-Based Debugging (Logical bug)

Before using complex debugging tools, simple program flow and state inspection can be performed using output

statements (printf).

Basic Value Tracing

```
printf("DEBUG: a = %d, b = %d\n", a, b);
```

Example: Tracking Loop Execution (sample3.c)

The following program adds numbers from 1 to 4 instead of 1 to 5 due to a logical bug.

Debug `printf()` statements help expose the exact iteration behavior.

```
#include <stdio.h>

int main(void)
{
    int i, sum = 0;
    for(i = 0; i < 5; i++)
    {
        printf("DEBUG: entering iteration i = %d\n", i);
        sum = sum + i;
        printf("DEBUG: sum updated to = %d\n", sum);
    }
    printf("Final Sum = %d\n", sum);
    return 0;
}
```

3. Introduction to GDB (GNU Debugger)

Background: What is GNU & GDB?

- GNU: A recursive acronym for "GNU's Not Unix!". It is an open-source operating system project initiated

by Richard Stallman in 1983 to provide a completely free Unix-like system.

- **GDB (GNU Debugger):** The standard debugger for the GNU software system, allowing developers to

inspect variable values, set execution breakpoints, and analyze crashes.

Installation & Compilation for GDB

```
# Update package list and install GDB on Linux systems
```

```
$ sudo apt update
```

```
$ sudo apt install gdb
```

```
# Compile source code with debugging symbols enabled (-g)
```

```
$ cc -g -Wall -Wextra program.c -o program
```

```
# Launch GDB with the compiled binary
```

```
$ gdb ./program
```

Quick Reference: Common GDB Commands

Command

Shorthand

Description / Purpose

`break <line / func>`

`b`

Sets a breakpoint at a specified line number or function name.

`run`

`r`

Starts execution of the program from the beginning.

`next`

`n`

Executes the next line of code (steps over function calls).

step

s

Executes the next line of code (steps into function calls).

print <var>

p

Prints the current value of a variable or expression.

`display <var>`

`disp`

Automatically prints the value of a variable after every step.

`continue`

`c`

Resumes execution until the next breakpoint or program completion.

`list`

`l`

Prints source code lines around the current execution position.

`info locals`

`info loc`

Lists all local variables and their current values in the current frame.

`backtrace`

`bt`

Displays the current call stack (chain of function calls).

`info breakpoints`

`i b`

Displays all active breakpoints and their numbers.

`delete <id>`

`d`

Deletes the specified breakpoint number (or all if omitted).

`disable <id>`

`dis`

Temporarily disables a breakpoint without deleting it.

`enable <id>`

`en`

Re-enables a previously disabled breakpoint.

`quit`

`q`

Exits the GDB debugging session.

Case 4: Debugging a Logical Error using GDB

Example: Code with Logic Bug (sample4.c)

```
#include <stdio.h>
```

// Line 1

```
int main()
```

// Line 2

```
{
```

// Line 3

```
int i, sum = 0;
```

// Line 4

```
for(i = 1; i < 5; i++)
```

// Line 5 (Logical Bug: should be $i \leq 5$)

```
{
```

// Line 6

```
sum = sum + i;
```

// Line 7

```
}
```

// Line 8

```
printf("Sum = %d\n", sum);
```

// Line 9

```
return 0;
```

// Line 10

```
}
```

- Expected Output: 15 (1 + 2 + 3 + 4 + 5)
- Actual Output: 10 (1 + 2 + 3 + 4)

Debugging Walkthrough in GDB

```
$ cc -g sample4.c -o sample4
```

```
$ gdb ./sample4
```

```
(gdb) list
```

```
(gdb) break 7
```

```
Breakpoint 1 at 0x1149: file sample4.c, line 7.
```

```
(gdb) run
```

```
Starting program: ./sample4
```

```
Breakpoint 1, main () at sample4.c:7
```

```
7
```

```
sum = sum + i;
```

```
(gdb) print i
```

```
$1 = 1
```

Page 5 of 9

```
(gdb) display i
```

1: i = 1

```
(gdb) display sum
```

2: sum = 0

```
(gdb) continue
```

Continuing.

Breakpoint 1, main () at sample4.c:7

7

```
sum = sum + i;
```

1: i = 2

2: sum = 1

```
(gdb) continue
```

...

1: i = 4

2: sum = 6

```
(gdb) continue
```

Sum = 10

[Inferior 1 (process) exited normally]

Conclusion: The loop terminates when i reaches 5 without executing the loop body for 5. The loop condition

should be changed from $i < 5$ to $i \leq 5$.

Case 5: Debugging Functions (next vs. step)

- next: Executes the line. If the line contains a function call, the function executes entirely in the background

without entering its body.

- step: Steps into the function call, transferring GDB's focus into the function body for line-by-line

inspection.

Example Code (sample5.c)

```
#include <stdio.h>

int square(int x)
{
    int result1;
    result1 = x * x;
    return result1;
}

int main()
{
    int result, n = 5;
    result = square(n);
    printf("Square = %d\n", result);
    return 0;
}
```

Option A: Stepping Over with next

```
(gdb) break main
```

```
(gdb) run
```

```
(gdb) next
```

```
# Moves to result = square(n);
```

```
(gdb) next
```

```
# Executes square(n) completely
```

```
(gdb) print result
```

```
$1 = 25
```

```
# Result obtained without stepping inside square()
```

Option B: Stepping Into with step

```
(gdb) break main
```

```
(gdb) run
```

```
(gdb) next
```

```
# Moves to result = square(n);
```

```
(gdb) step
```

```
# Enters square(x) function body!
```

square (x=5) at sample5.c:5

5

```
result1 = x * x;
```

```
(gdb) print x
```

```
$1 = 5
```

```
(gdb) next
```

```
(gdb) print result1
```

```
$2 = 25
```

Case 6: Debugging Runtime Errors using Backtrace (backtrace / bt)

When a program crashes (e.g., floating point exception / division by zero), GDB can pinpoint the exact function

call sequence leading to the crash.

Example Code (sample6.c)

```
#include <stdio.h>
```

```
int divide(int a, int b)
```

```
{
```

```
    return a / b;
```

```
// Division by zero when b = 0
```

```
}
```

```
int calculate(int x)
```

```
{
```

```
    return divide(x, 0);
```

```
// Passes 0 as divisor
```

```
}
```

```
int main()
```



```
{  
  
int result = calculate(10);  
  
printf("Result: %d\n", result);  
  
return 0;  
  
}
```

GDB Backtrace Execution

```
$ cc -g sample6.c -o sample6
```

```
$ gdb ./sample6
```

```
(gdb) run
```

```
Program received signal SIGFPE, Arithmetic exception.
```

```
0x000055555555132 in divide (a=10, b=0) at sample6.c:5
```

```
5
```

```
return a / b;
```

```
(gdb) backtrace
```

```
#0 0x000055555555132 in divide (a=10, b=0) at sample6.c:5
```

```
#1 0x000055555555151 in calculate (x=10) at sample6.c:10
```

```
#2 0x00005555555516a in main () at sample6.c:15
```

Understanding Stack Frames

- Frame #0: The function where the crash actually occurred (divide).
- Frame #1: The function that called frame #0 (calculate).
- Frame #2: The top-level function that started the call sequence (main).

You can inspect specific frames using the frame command:

```
(gdb) frame 0
```

```
(gdb) print b          # Shows b = 0, identifying division by zero
```

```
(gdb) frame 1
```

```
(gdb) print x          # Shows x = 10
```

Case 7: Debugging Array Out-of-Bounds Errors using GDB

In C, array bounds are not checked at runtime automatically. Accessing indices beyond the declared limit results

in undefined behavior or memory corruption.

Example Code (sample7.c)

```
#include <stdio.h>

int main()

{

int a[5];

int i;

for(i = 0; i <= 5; i++)

{

scanf("%d", &a[i]);
```

//Line 8

```
}

return 0;

}
```

Inspection in GDB

```
(gdb) break 8
```

```
(gdb) run
```

```
(gdb) print &a[0]
```

```
$1 = (int *) 0x7fffffff9c0
```

```
(gdb) print &a[4]
```

```
$2 = (int *) 0x7fffffff9d0
```

```
(gdb) print &a[5]
```

```
$3 = (int *) 0x7fffffff9d4    # Valid array size is 5 (indices 0..4). Index 5  
is out of bounds!
```

- Correct Loop Condition: `for(i = 0; i < 5; i++)`

Case 8: Segmentation Faults (SIGSEGV) using GDB

A Segmentation Fault occurs when a program attempts to access a memory location that it does not have

permission to access (such as dereferencing a NULL pointer).

Example Code (sample8.c)

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
int *p = NULL;
```

```
// Pointer points to address 0x0
```

```
*p = 10;
```

```
// Attempting to write to address 0x0 causes Segmentation Fault
```

```
printf("%d\n", *p);
```

```
return 0;
```

```
}
```

Debugging in GDB

```
$ cc -g sample8.c -o sample8
```

```
$ gdb ./sample8
```

```
(gdb) run
```

Program received signal SIGSEGV, Segmentation fault.

0x00005555555555139 in main () at sample8.c:6

6

```
*p = 10;
```

```
(gdb) print p
```

```
$1 = (int *) 0x0
```

```
(gdb) print *p
```

Cannot access memory at address 0x0

Corrected Code

```
#include <stdio.h>

int main()
{
    int x;

    int *p = &x;    // Assign p to point to valid memory address of x

    *p = 10;        // Valid write

    printf("%d\n", *p);

    return 0;
}
```

4. Systematic Debugging Strategy

To debug effectively, follow this 8-step methodology:

1. Observe Expected vs. Actual Output

2. Reproduce the Issue Consistently
3. Compile with Flags (-Wall -Wextra -g)
4. Isolate Suspicious Code Sections
5. Inspect State (printf or GDB breakpoints)
6. Apply a Single Minimal Fix
7. Recompile and Re-test
8. Verify with Edge Cases & Boundary Inputs

5. Exercises & Homework Questions

1. Error Classification: Write a small C program snippet illustrating each of the following error types: Syntax

error, Compilation/Type error, Linker error, Runtime error, and Logical error.

1. 1. Syntax Error

A syntax error occurs when the C code does not follow the language's grammar rules.

```
#include <stdio.h>
```

```
int main() {  
    printf("Hello World") // Missing semicolon  
    return 0;  
}
```

Error: Missing ; after printf().

2. Compilation / Type Error

This occurs when incompatible data types are used or a compile-time rule is violated.

```
#include <stdio.h>
```

```
int main() {
```

```
int x = "Hello";

printf("%d", x);

return 0;

}
```

Error: A string is assigned to an int variable.

3. Linker Error

A linker error occurs when the compiler cannot find the definition of a function or variable being used.

```
#include <stdio.h>
```

```
void display(); // Function declared but not defined
```

```
int main() {

    display();

    return 0;

}
```

Error: display() is called, but its definition is missing, causing an **undefined reference** linker error.

4. Runtime Error

A runtime error occurs while the program is executing.

```
#include <stdio.h>
```

```
int main() {

    int a = 10, b = 0;

    printf("%d", a / b);

    return 0;

}
```

Error: Division by zero can cause a runtime failure (undefined behavior).

5. Logical Error

A logical error occurs when the program runs successfully but produces an incorrect result.

```
#include <stdio.h>
```

```
int main() {  
    int a = 10, b = 5;  
    int sum = a - b; // Should be a + b  
  
    printf("Sum = %d", sum);  
    return 0;  
}
```

Error: The program executes without a compiler error, but it calculates 5 instead of the correct sum 15.

2. GNU Concept: Explain what the GNU project is and how GDB fits into the GNU ecosystem.

The GNU Project is a free and open-source software project started by Richard Stallman in 1983. GNU stands for “GNU’s Not Unix.”

Its main goal is to develop a complete Unix-like operating system and related software that users are free to **run, study, modify, and share**.

Some important GNU tools include:

1. **GCC (GNU Compiler Collection)** — compiles programs written in languages such as C and C++.
2. **GNU Make** — automates the process of building programs.
3. **GDB (GNU Debugger)** — helps programmers find and fix errors in programs.
4. **GNU Core Utilities** — provides common command-line utilities such as `ls`, `cp`, and `mv`.
5. **Bash** — a popular GNU command-line shell.

How does GDB fit into the GNU ecosystem?

GDB (GNU Debugger) is a part of the GNU software collection. It is used to **debug programs** by allowing programmers to examine what happens while a program runs.

With GDB, you can:

6. Set **breakpoints** to pause program execution.
7. Execute a program **step by step**.
8. Examine the values of **variables**.
9. Inspect the **call stack**.
10. Find where a program **crashes**.
11. Investigate logical and runtime problems.

For example, a C program can be compiled with GCC using debugging information:

```
gcc -g program.c -o program
```

Then it can be opened in GDB:

```
gdb ./program
```

Thus, **GCC and GDB work together**: GCC compiles the C program and includes debugging information, while GDB uses that information to help the programmer analyze and debug the program.

In short: The GNU Project provides a large collection of free software development tools, and **GDB is the GNU debugger within that ecosystem**, commonly used alongside GCC for debugging C and other programs.

3. Research and contrast the differences between: Breakpoint, Watchpoint, Catchpoint, and Tracepoint.

1. 1. Breakpoint

A **breakpoint** tells GDB to stop the program when it reaches a specific point in the code.

```
int main() {  
    int x = 10;
```

```
x = 20;          // Breakpoint can be placed here
printf("%d", x);
}
```

GDB command:

```
(gdb) break main
```

Purpose: Useful for examining the program's state at a particular line or function.

2. Watchpoint

A **watchpoint** monitors the value of a variable. GDB stops execution when that value changes.

```
int x = 10;
x = 20;    // Watchpoint triggers here
```

GDB command:

```
(gdb) watch x
```

Purpose: Particularly useful when you know **which variable is being changed incorrectly**, but don't know where the change occurs.

3. Catchpoint

A **catchpoint** stops execution when a specific **event** happens rather than simply reaching a line of code.

For example:

```
(gdb) catch throw
```

This tells GDB to stop when a C++ exception is thrown.

Other examples include:

```
(gdb) catch signal
(gdb) catch syscall
```

Purpose: Useful for investigating events such as signals, exceptions, system calls, or other runtime events supported by GDB.

4. Tracepoint

A **tracepoint** is designed to collect information about program execution, often without stopping the program at each event. It is especially useful for analyzing program behavior over time.

Example:

```
(gdb) trace main
(gdb) actions
> collect x
> end
```

The tracepoint can record selected information when execution reaches `main`.

Purpose: Useful when stopping the program would interfere with its behavior or when you want to collect information from many points during execution.

Key Difference

A simple way to remember them is:

- **Breakpoint** → "Stop here."
- **Watchpoint** → "Stop when this value changes."
- **Catchpoint** → "Stop when this event happens."
- **Tracepoint** → "Record what happens here."

So, the main distinction is **what causes the debugger to react**: a location, a data change, an event, or an observation/recording point.